

Object-Oriented Programming with Java

Weeks 1–2: Programming Foundations, Control Flow, Methods, Classes and Objects

BS Computer Science — Second Semester

Lecture Slides and Student Handout

How to Use This Deck

For the Instructor

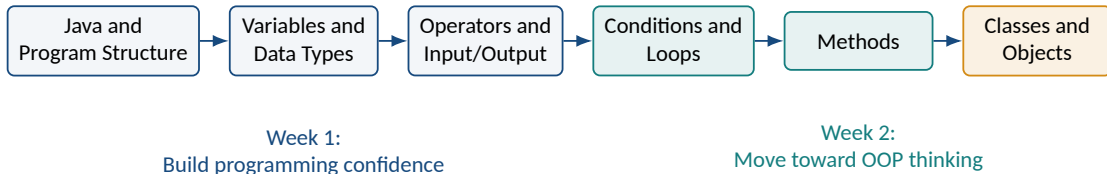
- Explain each visual before showing code.
- Ask students to predict program output.
- Trace variables on the board.
- Modify examples live during class.
- Connect every topic to a lab problem.

For Students

- Type the code instead of copying screenshots.
- Run small experiments after every concept.
- Read compiler errors carefully.
- Draw flowcharts before difficult problems.
- Practice regularly; programming is a skill.

Concept → Analogy → Code → Trace → Modify → Problem → Lab

Two-Week Learning Journey



Guiding Principle

Do not begin with advanced OOP theory. First make students comfortable with Java syntax and problem solving; then introduce objects as a natural way to organize programs.

Week 1 Learning Outcomes

By the end of Week 1, students should be able to:

- explain what programming is and why Java is used;
- distinguish the roles of the JDK, JVM, and Java runtime;
- write, compile, and execute a basic Java program;
- declare variables using appropriate data types;
- build expressions with arithmetic, relational, and logical operators;
- accept keyboard input using `Scanner`;
- produce formatted output;
- solve small calculation and decision problems independently.

Week 1 outcome: students should stop being afraid of Java syntax.

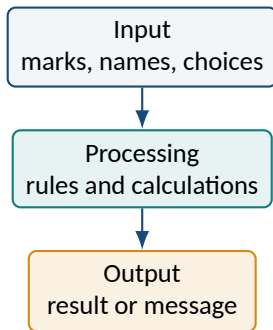
What Is Programming?

Simple Definition

Programming is the process of giving a computer a precise sequence of instructions to solve a problem or perform a task.

- A computer does not guess our intention.
- Instructions must be clear and ordered.
- Data enters the program.
- The program processes the data.
- A useful result is produced.

Example: marks → calculate percentage → display grade.



Programming Languages

Machine Language

- Binary instructions
- Hardware-specific
- Difficult for humans

Low-Level Languages

- Close to hardware
- Fast and controlled
- More difficult to write

High-Level Languages

- Human-readable syntax
- Easier development
- Java, Python, C++, C#



Why Java?

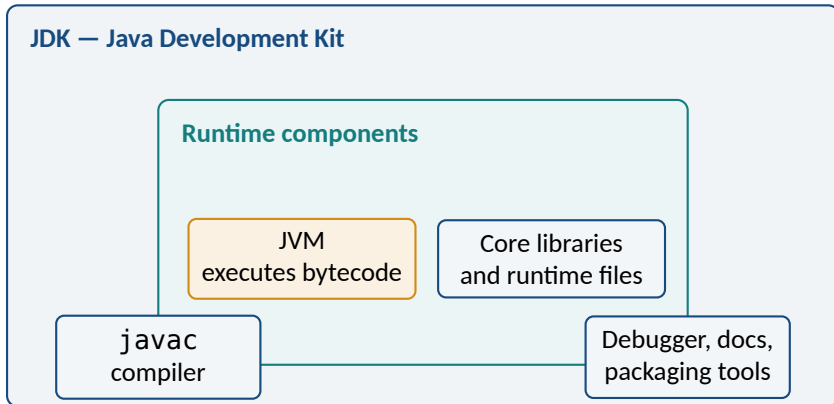
- **Platform independent:** compiled bytecode runs through a JVM.
- **Object-oriented:** programs can model real entities as objects.
- **Strongly typed:** variables have declared types.
- **Robust:** compile-time checking and managed memory reduce common errors.
- **Secure:** controlled runtime environment and mature security model.
- **Large ecosystem:** tools, libraries, frameworks, and community support.

Where Java Is Used

- Enterprise applications
- Web backends
- Android and mobile foundations
- Banking and business systems
- Desktop tools
- Teaching programming and OOP

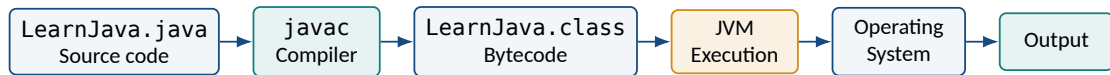
Java source is compiled once into bytecode; a suitable JVM executes that bytecode on each platform.

JDK, JVM, and Runtime Environment



- The **JDK** is needed to develop and compile Java programs.
- The **JVM** is the virtual machine that executes Java bytecode.
- Runtime libraries provide classes used by Java programs.

How a Java Program Executes



Compile

```
javac LearnJava.java
```

Checks syntax and creates bytecode.

Run

```
java LearnJava
```

Starts the JVM and executes the class.

Class question: If Java is compiled, why is it called platform independent?

Your First Java Program

```
1 public class LearnJava {  
2     public static void main(String[] args) {  
3         System.out.println("Hello World");  
4     }  
5 }
```

Important Rules

- File name: `LearnJava.java`
- Public class name must match the file name.
- Execution starts from `main()`.
- Statements normally end with a semicolon.
- Braces define blocks.

The proper entry point is **`public static void main(String[] args).`**

Anatomy of the First Program

Part	Meaning
public	The class or method can be accessed by the Java runtime.
class	Declares a class, which is a basic Java program structure.
LearnJava	The programmer-defined class name.
static	The method belongs to the class and can run before an object is created.
void	The method returns no value.
main()	The standard starting method of a Java application.
String[] args	An array that can receive command-line text arguments.
System.out	The standard output stream.
println()	Prints a value and moves to a new line.

Statements, Comments, and Naming

```
1 public class StudentInfo {  
2     public static void main(String[] args) {  
3         // This is a single-line comment  
4         String studentName = "Ayesha";  
5         int semester = 2;  
6  
7         System.out.println(studentName);  
8         System.out.println(semester);  
9     }  
10 }
```

Naming Conventions

- Classes: StudentInfo
- Variables: studentName
- Methods: calculateTotal()
- Constants: MAX_MARKS
- Use meaningful names.

Common Error Categories

Syntax Error

The code violates Java grammar.
`println("Hi")`
Missing semicolon.

Runtime Error

The program compiles but fails while running.
Example: invalid array position or division by zero.

Logic Error

The program runs but produces an incorrect result.
Example: using `+` instead of `*`.

Debugging Habit

Read the first compiler error carefully. Later errors may only be consequences of the first mistake.

Classroom Activity: Break the Program

Ask students to predict the result of each change:

- 1 Rename the file but keep the public class name unchanged.
- 2 Change `main` to `Main`.
- 3 Remove `static` from `main()`.
- 4 Remove a semicolon.
- 5 Remove one closing brace.
- 6 Change `println` to `print`.
- 7 Change `"Hello World"` to `'Hello World'`.

Students learn syntax faster when they deliberately create, read, and fix errors.

Variables: Named Storage Locations

A Variable Has

- a **name**;
- a **type**;
- a stored **value**;
- a scope and lifetime.

```
int marks = 85;
```

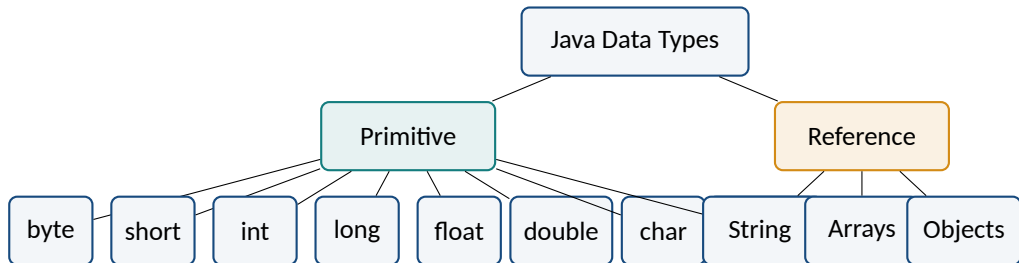
- `int`: type
- `marks`: name
- `85`: value

Variable: marks

85

type: int

Java Data Types: Big Picture



Important

`String` is widely used, but it is not a primitive type; it is a class.

Primitive Data Types

Type	Example	Typical Use
byte	<code>byte age = 20;</code>	Very small whole numbers
short	<code>short year = 2026;</code>	Small whole numbers
int	<code>int marks = 85;</code>	Default whole-number type
long	<code>long pop = 1000000L;</code>	Large whole numbers
float	<code>float price = 10.5f;</code>	Decimal with lower precision
double	<code>double salary = 50000.50;</code>	Default decimal type
char	<code>char grade = 'A';</code>	One Unicode character
boolean	<code>boolean passed = true;</code>	True/false condition

```
String name = "Ali"; // String is a reference type
```

Declaration, Initialization, and Assignment

```
1 // Declaration
2 int age;
3
4 // Assignment
5 age = 20;
6
7 // Re-assignment
8 age = 21;
```

```
1 // Declaration + initialization
2 int marks = 85;
3
4 // A constant
5 final double PI = 3.14159;
```

Type Compatibility

`int count = 12;` is valid, but `int count = 12.5;` is not. Java checks whether a value is compatible with the variable's declared type.

Operators Overview

Category	Operators	Purpose
Arithmetic	+ - * / %	Perform mathematical calculations
Relational	> < >= <= == !=	Compare two values; result is boolean
Logical	&& !	Combine or reverse boolean conditions
Assignment	= += -= *= /=	Store or update a value
Increment/decrement	++ --	Increase or decrease by one

An expression combines values, variables, and operators to produce a new value.

Expressions and Integer Division

```
1 int x = 10;  
2 int y = 3;  
3  
4 System.out.println(x + y); // 13  
5 System.out.println(x * y); // 30  
6 System.out.println(x / y); // 3  
7 System.out.println(x % y); // 1
```

Why Is $10 / 3$ Equal to 3?

Both operands are integers, so Java performs integer division and removes the fractional part.

Obtain a Decimal Result

```
double result = 10.0 / 3;  
Result: approximately 3.3333
```

Building Useful Calculations

```
1 double radius = 4.5;  
2 double area = 3.14159 * radius * radius;  
3  
4 double celsius = 30;  
5 double fahrenheit = (celsius * 9 / 5) + 32;
```

```
1 double obtained = 425;  
2 double total = 500;  
3 double percentage = obtained / total * 100;  
4  
5 double interest = principal * rate * time / 100;
```

Student Practice

Translate each formula into Java, test it with sample values, and manually verify the output.

Keyboard Input with Scanner

```
1 import java.util.Scanner;
2
3 public class UserInput {
4     public static void main(String[] args) {
5         Scanner input = new Scanner(System.in);
6
7         System.out.print("Enter your name: ");
8         String name = input.nextLine();
9
10        System.out.print("Enter your age: ");
11        int age = input.nextInt();
12
13        System.out.println(name + " is " + age + " years old.");
14        ;
15        input.close();
16    }
17 }
```

Common Scanner Methods

- `nextInt()`
- `nextDouble()`
- `next()`
- `nextLine()`
- `nextBoolean()`

Input-Process-Output in Java



Example

Enter marks: → user enters 85 → `nextInt()` converts text to an integer → marks stores 85.

The nextInt() + nextLine() Issue

```
1 int age = input.nextInt();  
2 String name = input.nextLine();
```

Problem

`nextInt()` leaves the newline character in the input buffer. The next `nextLine()` may read that leftover newline.

```
1 int age = input.nextInt();  
2 input.nextLine(); // consume newline  
3  
4 String name = input.nextLine();
```

Solution

Consume the leftover newline before reading a complete line.

Week 1 Lab Tasks

Basic Programs

- 1 Hello World
- 2 Student information
- 3 Addition of two numbers
- 4 Average of three numbers
- 5 Area of a rectangle

Calculation Programs

- 6 Area of a circle
- 7 Celsius to Fahrenheit
- 8 Simple interest
- 9 Percentage calculator
- 10 Basic grade calculator

Lab Rule

Students should type, compile, test with multiple inputs, and explain the result to a partner.

Student Marks Program

Create a Java program that asks for:

- the student's name;
- marks in five subjects;
- total possible marks.

The program calculates and displays:

- total obtained marks;
- percentage;
- student name and result summary.

Extension: Add a simple pass/fail decision after learning `if - else` in Week 2.

Week 1 Diagnostic Quiz

- 1 What are the roles of the JDK and JVM?
- 2 Why can Java bytecode run on different operating systems?
- 3 What is the purpose of `main()`?
- 4 What is the difference between `int` and `double`?
- 5 What does `10 / 3` produce when both values are integers?
- 6 Is `String` a primitive type?
- 7 Which Scanner method reads an entire line?
- 8 Write a program that reads two numbers and prints their sum.

Recommended duration: 10–15 minutes. Use it diagnostically, not only for marks.

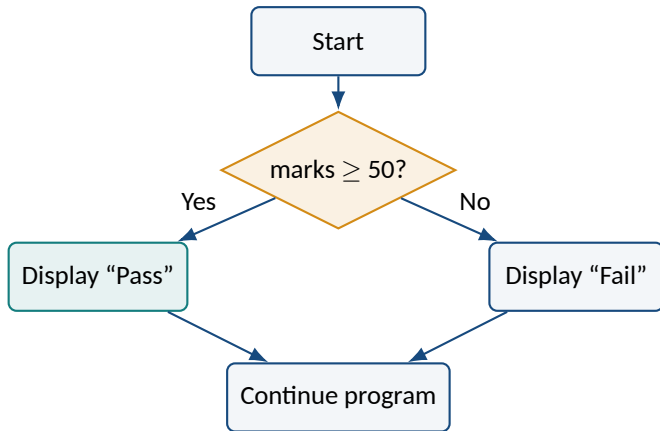
Week 2 Learning Outcomes

By the end of Week 2, students should be able to:

- make decisions using `if`, `if-else`, `else-if`, and `switch`;
- repeat operations using `for`, `while`, and `do-while`;
- use `break` and `continue` appropriately;
- define and call simple methods;
- explain parameters, arguments, return values, and local scope;
- identify objects in a real-world problem;
- distinguish a class from an object;
- create basic Java classes and instantiate multiple objects.

Week 2 bridges procedural problem solving and object-oriented thinking.

Decision Making: The Basic Idea



A condition is an expression that evaluates to either `true` or `false`.

if and if-else

```
1 if (marks >= 50) {  
2     System.out.println("Pass");  
3 }
```

Use `if` when an action is required only if a condition is true.

```
1 if (marks >= 50) {  
2     System.out.println("Pass");  
3 } else {  
4     System.out.println("Fail");  
5 }
```

Use `if-else` when exactly one of two alternatives must run.

Multiple Decisions with `else - if`

```
1 if (marks >= 80) {  
2     System.out.println("Grade A");  
3 } else if (marks >= 70) {  
4     System.out.println("Grade B");  
5 } else if (marks >= 60) {  
6     System.out.println("Grade C");  
7 } else if (marks >= 50) {  
8     System.out.println("Grade D");  
9 } else {  
10     System.out.println("Grade F");  
11 }
```

Order Matters

Conditions are checked from top to bottom. The first true block runs, and the remaining blocks are skipped.

Common Mistake

Starting with `marks >= 50` would incorrectly classify higher marks before the program reaches A, B, or C conditions.

The switch Statement

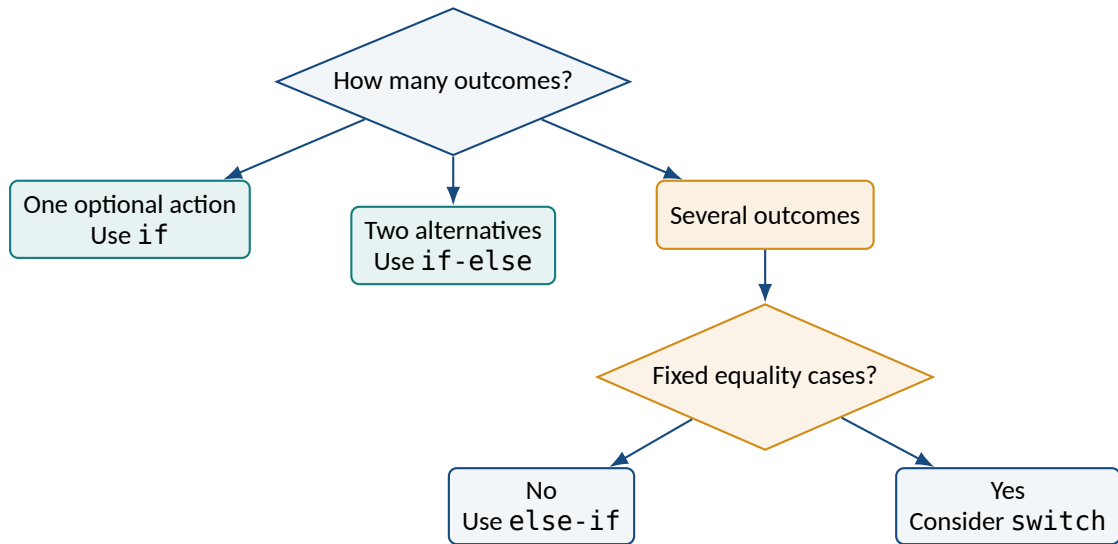
```
1 switch (choice) {  
2     case 1:  
3         System.out.println("Add");  
4         break;  
5     case 2:  
6         System.out.println("Subtract");  
7         break;  
8     case 3:  
9         System.out.println("Multiply");  
10        break;  
11    default:  
12        System.out.println("Invalid choice");  
13 }
```

Use switch When

- one expression is compared with several fixed values;
- a menu or option list is being processed;
- cases are clearer than a long equality-based `else-if` chain.

`break` prevents execution from continuing into the next case.

Selecting a Decision Structure



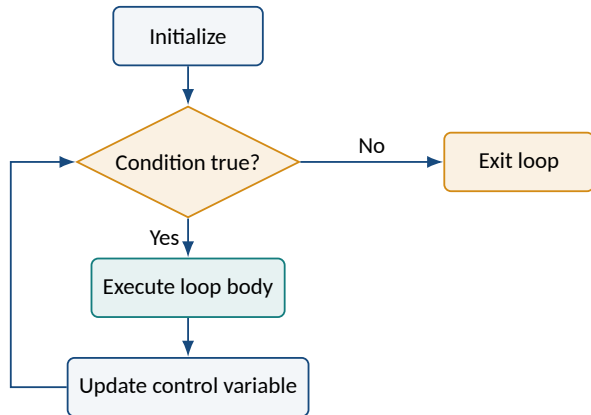
Why Do We Need Loops?

Without a loop:

- write the same statement many times;
- difficult to modify;
- impractical when repetition count is large or unknown.

With a loop:

- define the repeated action once;
- control when repetition begins and ends;
- process collections of values efficiently.

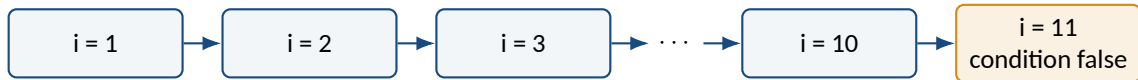


The for Loop

```
1 for (int i = 1; i <= 10; i++) {  
2     System.out.println(i);  
3 }
```

Three Parts

- 1 `int i = 1` — initialization
- 2 `i <= 10` — condition
- 3 `i++` — update



while and do-while

```
1 int i = 1;
2
3 while (i <= 10) {
4     System.out.println(i);
5     i++;
6 }
```

```
1 int i = 1;
2
3 do {
4     System.out.println(i);
5     i++;
6 } while (i <= 10);
```

while

Checks the condition before the body. It may execute zero times.

do-while

Executes the body first, so it always runs at least once.

Comparing the Three Loops

Loop	Best Used When	Condition Checked	Example
for	Number of repetitions is known or count-controlled	Before each iteration	Print 1-100
while	Repetition depends on a condition and count may be unknown	Before each iteration	Keep reading until valid input
do-while	The body must run at least once	After each iteration	Display a menu at least once

All three loops can solve many of the same problems; choose the one that communicates your intention most clearly.

break and continue

```
1 for (int i = 1; i <= 10; i++) {  
2     if (i == 6) {  
3         break;  
4     }  
5     System.out.println(i);  
6 }
```

`break` immediately exits the loop.

```
1 for (int i = 1; i <= 10; i++) {  
2     if (i == 6) {  
3         continue;  
4     }  
5     System.out.println(i);  
6 }
```

`continue` skips the current iteration and moves to the next one.

Loop Practice Problems

Students should implement and trace:

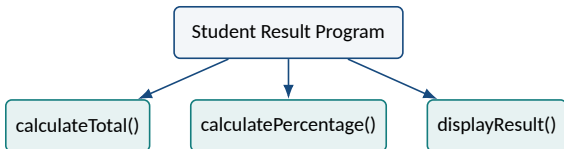
- 1 Print numbers from 1 to 100.
- 2 Print even numbers from 1 to 100.
- 3 Print odd numbers from 1 to 100.
- 4 Display a multiplication table.
- 5 Find the sum from 1 to n .
- 6 Calculate a factorial.
- 7 Reverse the digits of a number.
- 8 Count the digits of a number.
- 9 Check whether a number is prime.
- 10 Keep asking until valid input is entered.

Teaching tip: Require a variable-trace table for factorial, reverse-number, and prime-number problems.

Why Methods?

Methods help us:

- divide a large problem into smaller tasks;
- reuse logic without copying code;
- improve readability;
- test one task at a time;
- prepare for object behavior in OOP.



A method gives a meaningful name to a reusable piece of behavior.

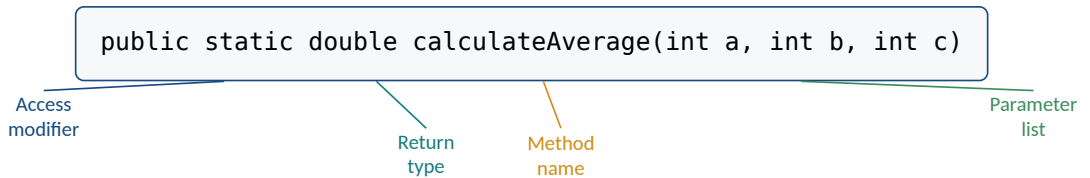
Defining and Calling Methods

```
1 public static void sayHello() {  
2     System.out.println("Hello");  
3 }  
4  
5 public static void main(String[] args) {  
6     sayHello();  
7 }
```

```
1 public static int add(int a, int b) {  
2     return a + b;  
3 }  
4  
5 public static void main(String[] args) {  
6     int sum = add(10, 5);  
7     System.out.println(sum);  
8 }
```

Method Anatomy

```
public static double calculateAverage(int a, int b, int c)
```



Access
modifier

Return
type

Method
name

Parameter
list

- A **parameter** is declared in the method definition.
- An **argument** is the actual value supplied during a call.
- `void` means the method does not return a value.

Parameters, Return Values, and Scope

```
1 public static double calculateAverage(  
2     int a, int b, int c) {  
3     double average = (a + b + c) / 3.0;  
4     return average;  
5 }  
6  
7 public static void main(String[] args) {  
8     double result = calculateAverage(70, 80, 90);  
9     System.out.println(result);  
10 }
```

Local Scope

average exists only inside
calculateAverage().
result exists only inside main().

Question

Can main() directly print the variable
average? Why not?

From Procedural Thinking to OOP Thinking

Procedural View

Focus on steps and functions:

- 1 read student data;
- 2 calculate total;
- 3 calculate percentage;
- 4 display result.

Object-Oriented View

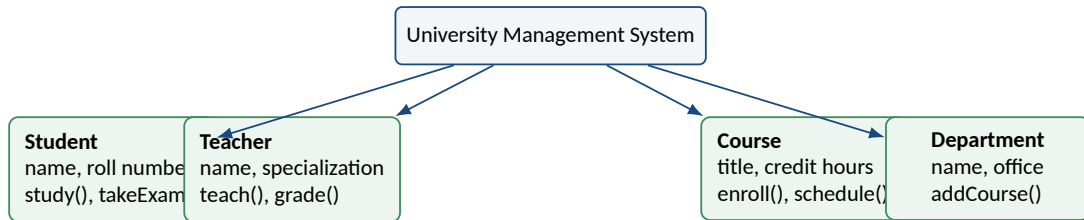
Focus on entities with data and behavior:

Student object

- stores name, roll number, and marks;
- calculates its total and percentage;
- displays its result.

OOP organizes related data and behavior together.

Finding Objects in a University System



Class discussion: What other objects exist? What information and actions belong to each object?

Object = State + Behavior + Identity

State

Information stored by the object.

- name
- roll number
- semester
- marks

Behavior

Actions the object can perform.

- study()
- attendClass()
- takeExam()
- displayResult()

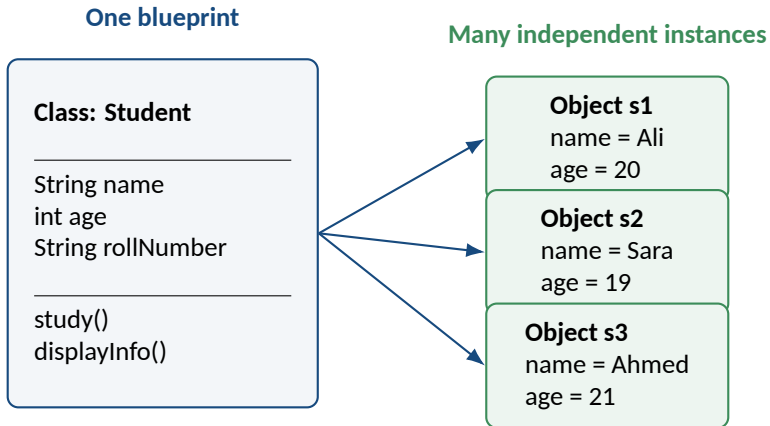
Identity

What makes one object distinct from another.

- Student s1
- Student s2
- Same class
- Separate objects

An object is not only data and not only methods; it combines both as one identifiable entity.

Class vs Object: Blueprint Analogy



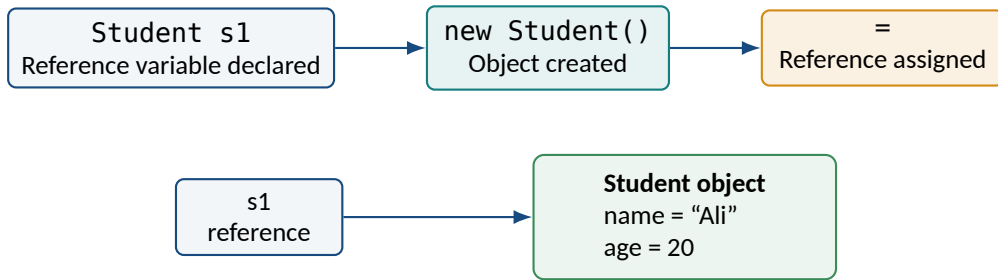
Creating the First Class

```
1 class Student {  
2     String name;  
3     int age;  
4  
5     void study() {  
6         System.out.println(  
7             name + " is studying.");  
8     }  
9 }
```

```
1 public class Main {  
2     public static void main(String[] args) {  
3         Student s1 = new Student();  
4  
5         s1.name = "Ali";  
6         s1.age = 20;  
7  
8         s1.study();  
9     }  
10 }
```

`new Student()` creates an object; `s1` stores a reference to that object.

What Happens During Object Creation?



The variable **s1** is not the complete object; it refers to the object created in memory.

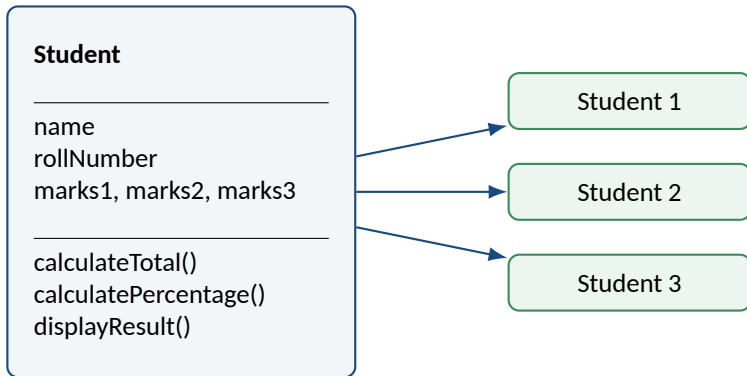
Week 2 Lab: Identify State and Behavior

Class	Suggested State	Suggested Behavior
Student	name, age, rollNumber, department	displayInfo(), study()
Car	brand, model, color, speed	accelerate(), brake(), displayInfo()
BankAccount	accountNumber, holderName, balance	deposit(), withdraw(), displayBalance()
Course	title, code, creditHours	enrollStudent(), displayCourse()
Teacher	name, subject, employeeId	teach(), displayInfo()

Student Task

For each class, create at least two objects, assign different values, and call the available methods.

Week 2 Mini Project: Student Result System



Main Learning Point

One class defines a reusable structure; many objects store separate student data and use the same behaviors.

Recommended Teaching Sequence

Week	Topic	Main Outcome
1	Java introduction	Understand Java and its execution model
1	Program structure	Write and run basic programs
1	Variables and data types	Store information correctly
1	Operators	Perform calculations and comparisons
1	Input/output	Build interactive programs
1	Basic problems	Develop confidence and debugging habits
2	Conditions	Make decisions
2	Loops	Repeat operations
2	Methods	Create reusable units of behavior
2	OOP introduction	Think in terms of entities
2	Classes and objects	Build first object-oriented programs
2	Mini project	Apply the concepts together

Topics to Delay Until Foundations Are Strong

Do not overload the first two weeks with:

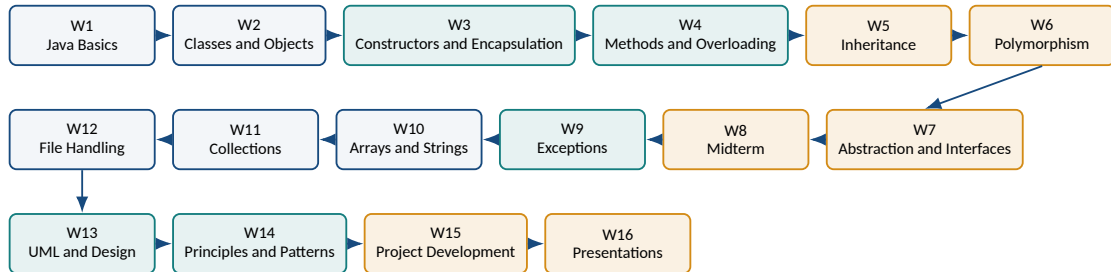
- inheritance;
- polymorphism;
- abstract classes;
- interfaces;
- generics;
- collections framework;

Also delay:

- advanced exception handling;
- design patterns;
- SOLID principles;
- advanced UML;
- multithreading;
- framework-specific development.

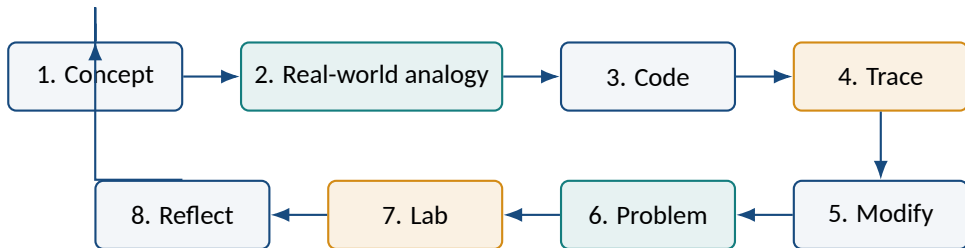
Students need a correct mental model before they need advanced terminology.

Suggested 16-Week OOP Course Roadmap



Note: Adjust the sequence to match departmental credit hours, lab structure, and assessment policy.

Recommended Teaching Cycle



Example Cycle

Class concept → student blueprint → Java class → trace object creation → add attributes → result-management problem → lab implementation.

Instructor Checklist for Weeks 1-2

Before Class

- Verify JDK and IDE installation.
- Prepare runnable code files.
- Test examples from a clean project.
- Prepare common-error demonstrations.
- Print or share lab instructions.

During Class

- Ask for predictions before running code.
- Trace values visually.
- Use student-generated examples.
- Pause for short coding tasks.
- Check understanding, not only completion.

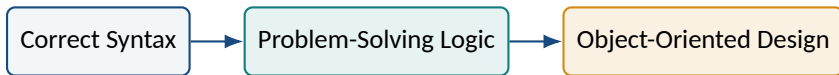
Student Self-Check

After two weeks, a student should be able to answer “yes” to these questions:

- Can I create, compile, and run a Java class?
- Can I explain the basic execution flow from source code to JVM output?
- Can I choose suitable variables and data types?
- Can I read input and display calculated output?
- Can I use conditions and loops to control execution?
- Can I write and call a method with parameters and a return value?
- Can I identify state and behavior in a real-world entity?
- Can I create a class and instantiate more than one object?

Learn Java by building, testing, and improving programs.

Classes and objects become meaningful when they solve a real problem.



Next topic: Constructors and encapsulation.